

Program 4

Project Structure:

```
lab4
├──
├── app.py
├── requirements.txt
├── Dockerfile
└── README.md
```

Dockerfile:

```
FROM python:latest

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "hello world from container"

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000)
```

Requirements.txt

```
Flask==2.3.3
```

Setup and Installation:

Docker build command:

```
$ sudo docker build -t program4 .
[+] Building 0.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 199B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 260B
=> [1/5] FROM docker.io/library/python:latest@sha256:1ad1a43b5
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:9726101fc3e2556a3e79677ba73114e1e82
```

Run the docker on port:

```
$ sudo docker run -p 5000:5000 program4
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
72.17.0.1 - - [07/Nov/2025 04:19:53] "GET / HTTP/1.1" 200 -
72.17.0.1 - - [07/Nov/2025 04:19:56] "GET / HTTP/1.1" 200 -
72.17.0.1 - - [07/Nov/2025 04:23:05] "GET / HTTP/1.1" 200 -
72.17.0.1 - - [07/Nov/2025 04:23:05] "GET /favicon.ico HTTP/1.1" 404 -
72.17.0.1 - - [07/Nov/2025 04:26:31] "GET / HTTP/1.1" 200 -
C
```

Open:

<http://localhost:5000>



hello world

Now, log in to you github and generate pat:

How to Generate a Personal Access Token (Classic)

Log in to your GitHub account.

First create a new repo then,

Go to Settings → Developer settings → Personal access tokens → Tokens (classic)
or visit directly:

<https://github.com/settings/tokens>

Click Generate new token → Generate new token (classic)

Enter a note (e.g., *Git CLI Access*) and select expiration time.

Under Select scopes, check:

- repo → (Full control of private repositories)
- workflow → (If you use GitHub Actions)

Click Generate token

Step 1: Initialize a Local Repository

Create or move into your project directory:

```
cd ~/lab4demo
```

Initialize a new Git repository:

```
git init
```

Rename the default branch to main:

```
git branch -M main
```

Step 2: Add and Commit Files

```
git add .
```

Commit them with a message:

```
git commit -m "Initial commit"
```

Step 3: Connect to a Remote GitHub Repository

```
git remote add origin https://github.com/<your-username>/<repo-name>.git
```

Step 4. Using Username and PAT with Git

```
git push -u origin main
```

You will be prompted:

Username for 'https://github.com': <your username of github>

Password for 'https://kuntiwari1@github.com': <paste your PAT>

```
(singh@singh)-[~/lab4demo]
$ git push -u origin main

Username for 'https://github.com': kuntiwari1
Password for 'https://kuntiwari1@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 642 bytes | 642.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kuntiwari1/helllllo.git
    20b2d46..4dcc020  main -> main
branch 'main' set up to track 'origin/main'.

(singh@singh)-[~/lab4demo]
```

